

A programmer's walk through `selfupdate`

Audience: a new programmer who needs to change, diagnose, or run the trainer without weakening the experiment's strict block-local claim.

Scope and revision: this is an implementation guide for the `selfupdate\_lw` branch as of 2026-07-17. The code is the executable source of truth; `AGENTS.md` is the operational policy, and `docs/hidden\_loss.md`, `docs/runtime.md`, and `docs/training\_pipeline\_v3.md` are the detailed method contracts. This guide deliberately quotes the important training code so a reader can trace the control and gradient paths without first loading a model.

1. What this repository is—and is not

This branch trains a model to reproduce *teacher hidden states* at aligned token positions. The teacher sees a privileged RAG passage or thinking trace; the student sees the same prompt with that segment censored. It is not a final-logit distillation repository. It contains no active behavioral readout or reference-text training objective.

The single sentence to keep in mind while reading every training change is:

Each trainable block receives a detached student input, is compared with the frozen teacher state at that block, and can backpropagate only into that block's trainable parameters.

The vocabulary basis is frozen: embedding, final norm, and LM head never receive updates. `lens\_kl` is allowed only as a *local measurement* through that frozen basis. `CE-eval-loss` and `KL-eval-loss` are whole-training-set evaluation readings; they are never optimizer losses.

2. Map before editing

Start with this route through the tree:

```
configs/base.yaml + configs/experiments/<arm>.yaml
  |
  v
scripts/train.py          CLI, import-path pin, config load
  |
  v
train/layerwise.py        dispatch and v1/v2 schedule loops
  |                        (or train/online_v3.py for pipeline 3)
+--> train/validate.py    reject illegal knob combinations
+--> train/runtime.py     load/place/freeze model; teacher/cache; save
+--> train/steps.py       local block forward/backward primitives
+--> train/losses.py      HiddenLoss construction and metric math
+--> data/dataset.py      masks, aligned rows, batches/grid tiles
+--> train/telemetry.py   run JSONL and evaluation-only probes
  |
  v
runs/<run-name>/          config, metrics.jsonl, checkpoint, reports
```

The small modules are intentional. Do not collapse them by putting model loading into a schedule or by making a loss helper construct an optimizer: the split prevents an execution-policy change from silently becoming a new scientific method.

Useful entry points:

- `scripts/train.py` — ordinary v1/v2 invocation.
- `scripts/l40s\_train\_v3.sh` — coordinated L40S launcher for pipeline v3.
- `scripts/audit\_configs.py` — validates every committed experiment overlay.
- `scripts/train\_certify.py` — mint temporary before/after numerical fingerprints when a change is intended to preserve numerics.
- `scripts/memory\_plan.py` — advisory placement/batch plan before a large model is loaded.

3. The first executable line: CLI and configuration

Every script pins imports to this checkout. That is not stylistic: the shared venv intentionally has no editable `selfupdate` install, so a script must not accidentally import a sibling checkout.

```
# scripts/train.py
sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "src"))

from selfupdate.utils.env import cap_cpu_threads
cap_cpu_threads()

from selfupdate.config import load_config
from selfupdate.train.layerwise import train_layerwise

cfg = load_config(args.config, args.experiment)
if cfg.train.method != "layerwise":
    sys.exit(f"unsupported train.method {cfg.train.method!r}; use 'layerwise'")
run_dir = train_layerwise(cfg)
```

An experiment is `configs/base.yaml` plus a small overlay. Defaults are scientific variables, so a reproducing overlay pins every distinguishing knob. Important fields live in `ExperimentConfig` and its `ModelConfig`, `DataConfig`, `MaskConfig`, `CacheConfig`, and `TrainConfig` components in `src/selfupdate/config.py`.

Read `TrainConfig` as a protocol declaration, not a bag of performance flags. For example, these fields determine different execution laws:

```
# src/selfupdate/config.py (excerpt)
pipeline_version: int = 1
pp_execution: str = "serial"          # serial | wavefront | independent
update_granularity: str = "legacy_answer_sum"
trajectory_source: str = "student_hidden"
teacher_hidden_source: str = "online"
schedule: str = "summed"
online_optimizer: str = "adamw"      # immediate_sgd in pipeline 3
hidden_loss: str = "nmse"
conn_window: int = 1
```

For current Pareto bases, `conn\_window: 1` means one block-local objective at a time. Do not introduce `readout\_\*` config keys on this branch. If a new knob affects gradient flow, target source, masking, or vocabulary freezing, it belongs in validation before it belongs in an experiment YAML.

4. Dispatch: where a run becomes a method

`train\_layerwise` is the orchestration spine. Its first call is deliberately validation, before creating a run directory or loading weights:

```
# src/selfupdate/train/layerwise.py
def train_layerwise(cfg: ExperimentConfig) -> Path:
    _validate_knob_schedule(cfg)
    run_dir, log = setup_run_dir(cfg)
    seed_everything(cfg.train.seed)
    moe_load_kw = dequantize_overrides(cfg.model.name, cfg.train.moe_mode)
    rt = TrainingRuntime(cfg).load(moe_load_kw)
    tok, stack = rt.tokenizer, rt.stack
    ...
```

The pipeline-v3 fork happens early because it has a different atomic update event (one answer/token coordinate, not a conventional accumulated batch):

```
if cfg.train.pipeline_version == 3:
    teacher = rt.load_teacher(moe_load_kw)
    cache = rt.load_cache()
    log.log(kind="teacher_cache_source", ...)
    with cooperative_stop_signals():
        stopped = train_online_v3(cfg, stack, tok, log, cache, teacher=teacher)
        locality = certify_locality_resident(
            cfg, stack, tok, cache, run_dir, teacher=teacher)
        log.log(kind="locality_certification", **{
            key: locality[key] for key in (
                "items", "gradient_contract", "final_logit_training",
                "local_grad_norm", "cross_block_leak_grad_norm",
                "frozen_vocab_grad_norm", "local_signal_present_in_every_block",
                "passed"}})
    rt.save_checkpoint(run_dir)
```

For pipeline v1/v2 the dispatcher chooses exactly one named schedule. The important implication is that a schedule chooses **input/target timing**, while the local primitive chooses the gradient boundary:

```
if cfg.train.schedule == "summed":
    _train_summed(cfg, stack, cache, tok, log, teacher, moe,
        release_teacher=release_teacher)
elif cfg.train.schedule == "teacher_censored":
    if teacher is None:
        raise ValueError("teacher_censored needs full-sequence teacher states ...")
    if cfg.mask.compaction != "remove":
        raise ValueError("teacher_censored assumes compaction=remove ...")
    _train_teacher_censored(cfg, stack, tok, log, teacher)
elif cfg.train.schedule == "mixed":
    ...
elif cfg.train.schedule == "sequential":
    ...
else:
    raise ValueError(f"unknown layerwise schedule {cfg.train.schedule!r}")
```

`summed` is the ordinary strict-local student-trajectory walk. `sequential` trains a block to plateau using cached preceding activations. `teacher\_censored` uses censored teacher-stream inputs; it requires an online/frozen teacher and is independently trainable by layer. `mixed` selects between those streams. The active branch policy is strict local matching, not connected-window or readout experimentation.

5. Runtime: loading, placement, and the frozen vocabulary tripwire

Schedules do not call `from\_pretrained`, choose devices, attach LoRA, or make an optimizer. `TrainingRuntime` owns all of that. Its constructor resolves pipeline ownership and chooses the base dtype; `load()` sets the only legal model construction order:

```
# src/selfupdate/train/runtime.py
def load(self, moe_load_kw: dict | None = None) -> "TrainingRuntime":
    moe_load_kw = moe_load_kw or {}
    self.model = self._load_placed(self.student_src, self.base_dtype,
        **moe_load_kw)

    if self.cfg.train.lora.enabled:
        from .lora import attach_lora
        self.peft_model = attach_lora(self.model, self.cfg.train.lora)
        self.model = self.peft_model.get_base_model()

    self.model.train()
    self.stack = BlockStack(self.model,
```

```
        hook_free_walk=self.pp_map is not None)
    if self.pp_map is not None:
        _replicate_frozen_output_head_for_pp(self.stack)
    self.stack.freeze_non_blocks()
    self._vocab_sig0 = vocab_signature(self.stack)
    return self
```

``BlockStack.freeze_non_blocks()`` is where the model becomes a layerwise student. The stack exposes embeddings, decoder blocks, final norm, LM head, and a model-family adapter rather than letting schedule code poke at architecture-specific attribute paths. The sequence is crucial: attach LoRA first, set train mode, construct the adapter, then freeze everything outside the blocks. A frozen output-head replica may be placed on the last pipeline stage when a checkpoint has tied embedding/head weights; it remains frozen and is evaluation-only.

Before publishing a checkpoint, the runtime recomputes a cheap exact fingerprint of embedding/final-norm/head tensors:

```
def check_vocab_frozen(self) -> None:
    if vocab_signature(self.stack) != self._vocab_sig0:
        raise RuntimeError(
            "frozen-vocabulary violation: embedding/final-norm/head changed "
            "during training – refusing to save (docs/hidden_loss.md)")
```

Checkpoint publication itself is atomic. Consumers interpret ``runs/<name>/checkpoint`` as a complete, loadable dependency, not an in-flight directory:

```
target = run_dir / "checkpoint"
if target.exists():
    raise FileExistsError(f"refusing to replace existing checkpoint publication:
{target}")
staging = Path(tempfile.mkdtemp(prefix=".checkpoint.incomplete-", dir=run_dir))
try:
    if self.peft_model is not None:
        self.peft_model.save_pretrained(staging)
    else:
        self.model.to(torch.bfloat16)
        self.model.save_pretrained(staging)
    self.tokenizer.save_pretrained(staging)
    ...
    staging.rename(target)
except BaseException:
    shutil.rmtree(staging, ignore_errors=True)
    raise
```

Never replace this with direct writes into ``checkpoint``; a scheduler or evaluation worker could otherwise observe a partially saved model.

6. Teacher sources and the cache boundary

The teacher is frozen in one of two ways:

- A durable ``TeacherCache`` supplies aligned hidden-state target slices.
- ``OnlineTeacherSource`` uses adapters-off LoRA or a frozen model copy to

produce teacher states during the run.

The dispatch distinction matters. A cache normally stores only aligned targets; teacher-stream schedules need full teacher sequences and therefore require an online or frozen teacher. Pipeline v3 can use a node-local epoch-zero cache, materialized once under ``/dev/shm`` through its coordinated launcher.

The summed path expresses the normal cache/online decision plainly:

```
# src/selfupdate/train/layerwise.py, _train_summed (excerpt)
online = cfg.train.online_teacher
if not online and cache is None:
    raise ValueError("summed cached training needs a teacher cache")
ds = _make_dataset(cfg, cache, tok,
                  [] if online else list(range(1, n + 1)),
                  with_teacher_ids=online)
...
targets = (teacher.aligned_targets_batch(batch, device,
                                         capture_components=(cfg.train.hidden_loss == "component_nmse"))
          if online else batch.hidden)
```

`DistillDataset` owns alignment metadata. The central segmentation is `shared\_prefix | privileged | shared\_mid | answer`; the student removes or censors only the privileged section. The loss applies to the aligned `shared\_mid + answer` rows. Do not derive offsets ad hoc in a schedule—use the dataset's `Batch` fields (`s0`, `t0`, `A`, `aligned\_index`, masks, and teacher hidden mapping).

7. The strict-local summed walk, line by line

This is the most important function to understand: `\_summed\_batch` in `train/layerwise.py`. It constructs a student trajectory in increasing layer order, but it detaches the input at every layer boundary.

```
def _summed_batch(cfg, stack, loss_fn, batch: Batch, targets, device):
    n = stack.n_layers
    ids = batch.student_ids.to(device)
    pos = batch.position_ids.to(device)
    h = stack.embed(ids)
    pos_emb = stack.rope(h, pos)
    W = max(cfg.train.conn_window, 1)
    reduction = _update_reduction(cfg)
    layer_losses = []
    L = 1
    while L <= n:
        ...
        target = ((targets[L], targets[("attn", L)], targets[("mlp", L)])
                  if loss_fn.kind == "component_nmse" else targets[L])
        if L == n:
            loss_vals, h = last_block_step_batch(
                stack, h.detach(), pos_emb, target, batch, loss_fn,
                update_reduction=reduction,
            )
        else:
            loss_vals, h = local_block_step_batch(
                stack, L, h.detach(), pos_emb, target,
                batch, loss_fn, previous_target=targets.get(L - 1),
                update_reduction=reduction,
            )
        layer_losses.append(loss_vals)
        L += 1
    return layer_losses
```

Read the variable `h` carefully. It is the student's current hidden trajectory, so block `L` sees its own updated `h[L-1]`. But the call receives `h.detach()`. The returned output is also detached by the step primitive. Thus the forward values can influence later blocks, while the *autograd graph* cannot cross the block boundary. This is the intended forward-distillation geometry, not an accidental truncation.

For `conn\_window: 1`, the `W > 1` branches are skipped. The current branch's Pareto baseline lives in the last half of the function above. Treat historic window code as archival unless an owner explicitly

opens a different method.

8. The actual local backward primitive

`local\_block\_step\_batch` in `train/steps.py` is the mechanical enforcement of locality. It forwards exactly one block, computes a per-example loss from aligned rows, reduces it only as configured for the physical update, and calls backward before returning a detached output:

```
def local_block_step_batch(stack, L, h_in, pos_emb, target, batch: Batch, kind,
                           autocast=True, previous_target=None,
                           update_reduction="legacy_answer_sum"):
    loss_fn = HiddenLoss(kind) if isinstance(kind, str) else kind
    with torch.autocast(h_in.device.type, dtype=torch.bfloat16, enabled=autocast):
        if loss_fn.kind == "component_nmse":
            state_target, attn_target, mlp_target = target
            with _capture_block_components(stack, L) as components:
                h_out = stack.run_block(L, h_in, pos_emb)
                losses = _component_loss_per_example(
                    components, attn_target, mlp_target, batch)
            else:
                h_out = stack.run_block(L, h_in, pos_emb)
                losses = _layer_loss_per_example(
                    loss_fn, stack, L, h_out, h_in, target, previous_target, batch,
                )
            total = _reduce_example_losses(losses, batch, update_reduction)
        extra = pending_router_loss()
        (total if extra is None else total + extra).backward()
    return losses.detach(), h_out.detach()
```

The final block uses a separate primitive because its comparison must pass through the frozen final norm before measuring against the stored `h\_n` target:

```
def last_block_step_batch(stack, h_in, pos_emb, target, batch: Batch, kind, ...):
    n = stack.n_layers
    ...
    h_out = stack.run_block(n, h_in, pos_emb)
    normed = stack.final_norm(h_out)
    aligned = _gather_batch_rows(normed, batch.aligned_index)
    losses = _hidden_loss_per_example(
        loss_fn, aligned, target, batch.A.tolist(), normed=True, layer=n)
    total = _reduce_example_losses(losses, batch, update_reduction)
    ...
    total.backward()
    return losses.detach(), h_out.detach()
```

There is no `lm\_head(...)` in this normal loss path. If you add one, it must remain a frozen local metric (`lens\_kl`) and must preserve the detach boundary; it must never become a final-logit training target.

The aligned-row gather and loss reduction are also purposeful. Padded batches need an index gather, then individual valid lengths, rather than a single boolean-mask reduction that risks padding or host synchronization:

```
def _hidden_loss_per_example(loss_fn, student_h, teacher_h, lens, *, normed=False,
                             layer=None):
    teacher_h = teacher_h.to(student_h.device)
    losses = []
    for i, k in enumerate(lens):
        losses.append(loss_fn(student_h[i, :k], teacher_h[i, :k],
```

```
        normed=normed, layer=layer))
    return torch.stack(losses)

def _reduce_example_losses(losses, batch, update_reduction):
    if update_reduction == "answer_mean":
        return losses.mean()
    if update_reduction in ("token", "token_mean"):
        weights = batch.A.to(device=losses.device, dtype=losses.dtype)
        return (losses * weights).sum() / weights.sum().clamp_min(1)
    ...
```

`batching: item` is internally a B=1 padded batch and is intended to be bit-exact with the old item loop. Larger padded batches may differ in bf16 kernel rounding; that is why a numerics-preserving change is checked with the on-demand certification instrument rather than asserted by a stored fixture.

9. Loss construction: one supported gateway

Do not instantiate ad-hoc objectives inside a schedule. `HiddenLoss.from\_config` is the construction gate; it has access to the stack's frozen final norm/head and rejects incompatible loss/config combinations.

The `HiddenLoss` taxonomy is worth knowing:

- Geometric: `nmse`, `l2mse`, `cosine`, `huber`, and related residual-space measures.
- Vocabulary metric: `vocab\_mse`, sampled vocab cosine, `lens\_kl`, `lens\_js`.
- Increment: `delta\_\*`, which compare a block's raw update at interior layers and deliberately fall back to absolute state at cache boundaries.
- Component: `component\_nmse`, whose hooks capture recombined attention and MLP writes rather than attention probabilities.

The layer boundary treatment in `steps.py` makes the delta convention explicit:

```
def _layer_loss_per_example(loss_fn, stack, L, h_out, h_prev, teacher_h,
                           teacher_prev, batch):
    if loss_fn.is_delta and 1 < L < stack.n_layers:
        if teacher_prev is None:
            raise ValueError(
                f"{loss_fn.kind} at interior layer {L} needs h{L - 1} teacher target")
        return _delta_loss_per_example(
            loss_fn, _gather_batch_rows(h_out, batch.aligned_index),
            _gather_batch_rows(h_prev, batch.aligned_index),
            teacher_h, teacher_prev, batch.A.tolist())
    aligned = _gather_batch_rows(stack.loss_view(L, h_out), batch.aligned_index)
    return _hidden_loss_per_example(
        loss_fn, aligned, teacher_h, batch.A.tolist(),
        normed=(L == stack.n_layers), layer=L)
```

The relevant safety interpretation is: `stack.loss\_view()` knows whether a layer needs special representation handling; schedule code should not assume every model's raw decoder output has identical final-boundary semantics.

10. When parameters actually move

The summed schedule accumulates local gradients across the configured physical tile, logs the pending scalar data, then steps through `OptimizerPlan`:

```
# src/selfupdate/train/layerwise.py, _train_summed (excerpt)
if _update_boundary(cfg, accum, next_step):
    _flush_train_log(log, epoch=epoch, step=step, accum=accum,
                    pending=pending_losses, n_layers=n, ...)
```

```
if anchor is not None:
    a_ids, a_states = anchor[0].next()
    if cfg.train.anchor_hidden_weight > 0:
        anchor_trajectory_step(stack, a_ids, a_states,
                               cfg.train.anchor_hidden_weight)

plan.step()
step += 1
```

`OptimizerPlan.build` makes the state-placement choice named and auditable:

```
if cfg.train.lora.enabled and not offload:
    kind, foreach = "lora_fused", True
elif offload:
    kind, foreach = "full_offload", False
else:
    kind, foreach = "full_resident", False
...
if kind == "full_offload":
    optimizers = [torch.optim.AdamW(params, lr=cfg.train.lr, foreach=False)
                  for params in block_params.values()]
else:
    all_params = [p for params in block_params.values() for p in params]
    optimizers = [torch.optim.AdamW(all_params, lr=cfg.train.lr, foreach=foreach)]
```

All policies retain the experiment's per-block clipping law:

```
def step(self) -> None:
    for params in self.block_params.values():
        torch.nn.utils.clip_grad_norm_(params, 1.0, foreach=self.foreach)
    if self.kind == "full_offload":
        self._step_offload()
        return
    for opt in self.optimizers:
        opt.step()
        opt.zero_grad(set_to_none=True)
```

Changing from resident AdamW to offloaded AdamW is an execution/memory choice, not authorization to change clipping scope, update ordering, or loss scale.

11. Pipeline v3: token-local online learning is a separate loop

Pipeline v3 remains block-local but changes the atomic event to one answer token. It uses `immediate\_sgd`, has no gradient accumulation, and writes each block before the next token. This must not be casually merged with v1/v2 batch scheduling.

The v3 local operation begins by clearing only this block's gradients and detaching its input:

```
# src/selfupdate/train/online_v3.py
def _local_forward(cfg, stack, loss_fn, layer, h_in, pos_emb, position_ids,
                  target, row, *, flow_keep=None, cache=None,
                  causal_length=None, prepared_attention_mask=None):
    params = _clear_block_grads(stack, layer)
    h_in = h_in.detach()
    with torch.autocast(h_in.device.type, dtype=torch.bfloat16,
                       enabled=h_in.device.type == "cuda"):
        h_out = stack.run_block(layer, h_in, pos_emb,
                                position_ids=position_ids, flow_keep=flow_keep,
                                past_key_values=cache, use_cache=cache is not None,
```

```
        causal_length=causal_length,
        prepared_attention_mask=prepared_attention_mask)
    view = stack.loss_view(layer, h_out)[0, row]
    loss = loss_fn(view, target.to(view.device),
                   normed=(layer == stack.n_layers), layer=layer)
    return loss, h_out, params
```

Then it performs one local backward and immediate write. The returned state is the **pre-write** forward value, detached, so it is valid causal input for the next block at the same token but cannot carry a graph:

```
def _local_update(...):
    loss, h_out, params = _local_forward(...)
    if cfg.train.online_write_dispatch == "grad_ready":
        accumulator = _arm_grad_ready(params, cfg.train.lr)
        loss.backward()
        grad_norm = _finish_grad_ready(params, accumulator)
    else:
        loss.backward()
        grad_norm = _immediate_sgd(params, cfg.train.lr)
    if cache is not None:
        _detach_cache_layer(cache, layer - 1)
    return loss.detach(), grad_norm.detach(), h_out.detach()
```

Pipeline v3 may also dispatch several **disconnected** roots to autograd in one call. This is a scheduling optimization, not cross-layer credit:

```
def _finish_disconnected_token(cfg, losses, params_by_layer):
    # Roots have detached block inputs and disjoint parameter sets.
    torch.autograd.backward(losses)
    grad_norms = [value.detach() for value in
                  _immediate_sgd_token(params_by_layer, cfg.train.lr)]
    return [loss.detach() for loss in losses], grad_norms
```

For $K > 1$ stale windows, causal masks are mandatory inside a chunk. `_PreparedIntactCausal.window`` returns no mask only for `q=1``; otherwise it builds the K -by-prefix causal mask. Never optimize this away based solely on a single-token benchmark.

12. Validation is a scientific firewall

``validate_knob_schedule`` prevents settings that look syntactically plausible but violate an implemented contract. Pipeline v3 checks illustrate the approach:

```
# src/selfupdate/train/validate.py (excerpt)
if cfg.train.pipeline_version == 3:
    if cfg.train.update_granularity != "online":
        bad.append("pipeline_version=3 requires update_granularity=online")
    if sched != "summed":
        bad.append("pipeline-v3 online execution uses the summed forward layer walk")
    if cfg.train.grad_accum != 1:
        bad.append("pipeline-v3 requires grad_accum=1")
    if cfg.train.online_optimizer != "immediate_sgd":
        bad.append("pipeline-v3 requires online_optimizer=immediate_sgd")
```

The same validator rejects forbidden active methods, unsupported target/source combinations, and accidental readout behavior. Add a new rule here when a configuration could otherwise change the causal/gradient contract. Then run:

```
python scripts/audit_configs.py
```

before you launch anything.

13. Telemetry versus optimization

The hot loop avoids per-layer `.item()` and CPU copies. It keeps loss tensors on GPU in ``pending_losses``, then ``_flush_train_log`` aggregates and writes at an update boundary. This is both a performance rule and a reproducibility aid: do not introduce ``print``, ``cpu()``, or `.item()` inside a block walk.

At epoch boundaries, telemetry runs recall and standard-damage probes. The output-distance metrics have a special, non-negotiable interpretation:

```
CE-eval-loss / KL-eval-loss
= full-training-set, teacher-realized answer-token measurements
= collected once per completed epoch during normal traversal
= optimizer weight 0; never given to HiddenLoss/backward/AdamW
```

The identifiers are intentionally unlike ``lens_kl``. The latter can be a block-local training metric through frozen vocabulary coordinates; the former are output-level evaluation only. Any report or new telemetry code must keep this distinction explicit and record token/item coverage plus the zero optimizer weight.

14. A safe change workflow

Use this sequence for a trainer change:

- Identify whether the change touches execution only, target/trajectory semantics, masking/alignment, or gradient locality. Read the corresponding module plus ``docs/runtime.md``, ``docs/hidden_loss.md``, or ``docs/training_pipeline_v3.md`` before editing.
- Preserve the stack boundary: model loading/placement in ``runtime.py``, schedule ordering in ``layerwise.py``, local gradient mechanics in ``steps.py`` or ``online_v3.py``, and knob prohibitions in ``validate.py``.
- If it changes aligned spans, masks, cache layer conventions, or detaches, run ``python scripts/audit_configs.py``.
- For an intended numerical no-op, mint fresh temporary references from the current HEAD, change code, compare, then discard them:

```
```bash python scripts/train_certify.py --all --out-dir /tmp/$USER/certify_head # make the change
python scripts/train_certify.py --all --reference-dir /tmp/$USER/certify_head ```
```
- For a new scientific behavior, do not call it numerics-preserving. Pin every method-defining knob in a new overlay, check the locality certification record, and use the full-corpus reporting path.

Never ``pip install -e .`` into the shared venv. Run a repo script through its own ``src`` path guard; on an L40S use ``scripts/l40s_exec.sh scripts/train.py ...`` and on a compatible H100 use the container launcher. Read ``AGENTS.md`` before starting a GPU campaign: its cache staging, driver, scheduler, and reporting rules are part of a reproducible run.

## 15. A compact mental simulation

For one ordinary strict-local cached batch, simulate the code in this order:

1. Dataset supplies censored student IDs, position IDs, aligned indices, and frozen teacher `h[1] ... h[n]` slices.
2. `stack.embed(ids)` makes `h[0]`; RoPE is prepared from student positions.
3. At layer 1: `local_block_step_batch` receives `detach(h[0])`, creates a graph only through block 1, matches teacher `h[1]`, calls backward, returns `detach(h[1]_student)`.

4. At each interior layer  $L$ : repeat with `detach(student h[L-1])`; no graph reaches a shallower block, but the forward input reflects its current write.
5. At layer  $n$ : compare `frozen-final-norm(student h[n])` to teacher target; the head remains untouched.
6. After the configured physical tile: log pending tensors, clip per block, step the named optimizer policy, clear grads.
7. At epoch end: run evaluation-only output metrics, recall, damage, and parameter-delta telemetry; finally save only if the vocabulary signature matches.

If a proposed edit changes any arrow in that simulation—particularly a ``detach``, a target source, an aligned row selection, or the frozen vocabulary boundary—it is a method change until demonstrated otherwise. That is the right point to stop, update the config/validation/reporting contract, and ask for an explicit experimental decision rather than silently folding it into a performance patch.